



Trabalho de Projeto

4ª Fase

1 Descrição Geral

A componente teórico-prática da disciplina de sistemas distribuídos consiste no desenvolvimento de um projeto, dividido em 4 fases, utilizando a linguagem de programação C [1], sendo que a realização de cada uma das fases é necessária para a realização da fase seguinte. Por essa razão, **é muito importante que consigam ir cumprindo os objetivos de cada fase, de forma a não hipotecar as fases seguintes.**

O projeto consiste em implementar um sistema de gestão de inventário para um conjunto de um stand automóvel, utilizando uma lista encadeada [2] para armazenar a informação dos carros.

Na fase 1 foram definidas estruturas de dados e implementadas várias funções para lidar com a manipulação dos dados que vão ser armazenados na lista. Também foi construído um módulo para a serialização de dados, que teve como objetivo familiarizar os alunos com a necessidade de serializar os dados para a comunicação em aplicações distribuídas.

A fase 2 teve como objetivo implementar um sistema cliente-servidor simples, sendo o servidor responsável por manter uma lista (usando os módulos construídos no projeto 1) e sendo o cliente responsável por comunicar com o servidor para realizar operações na lista. Nesta fase, em vez de se utilizar o módulo de serialização desenvolvido na fase 1 (que teria de ser consideravelmente estendido com outras funções), foram utilizados os Protocol Buffers da Google [3], que permitem automatizar a criação das funções para serializar (codificar) e de-serializar (descodificar) os dados a serem enviados/recebidos, a partir de um ficheiro com a descrição destes dados.

Na 3ª fase do projeto foi criado um sistema concorrente que aceita e processa pedidos de múltiplos clientes em simultâneo através do uso de múltiplas *threads*. Este sistema também garante a gestão da concorrência no acesso aos dados partilhados no servidor, concretizando uma funcionalidade adicional para o registo de eventos num ficheiro de log.

Na 4ª e última fase do projeto iremos suportar tolerância a falhas através da replicação do estado do servidor, seguindo o modelo *Chain Replication* (Replicação em Cadeia) [4] e usando o serviço de coordenação ZooKeeper [5]. De forma genérica, vai ser preciso:

- Implementar a coordenação de servidores no ZooKeeper, de forma a suportar o modelo de replicação em cadeia (*Chain Replication*);
- Alterar o funcionamento do servidor para:
 - Procurar no ZooKeeper o servidor seguinte (sucessor) na cadeia de replicação;
 - Após executar uma operação sobre a lista (por exemplo `add`, `remove`, `get_by_year`, etc.), reenviar essa mesma operação para o servidor sucessor de forma a propagar a replicação;
 - Fazer watch no ZooKeeper de forma a ser notificado de alterações na cadeia de replicação e ligar-se ao seu novo sucessor, caso este tenha mudado.
- Alterar o funcionamento do cliente para:
 - Procurar no ZooKeeper os servidores que estão na cabeça e na cauda da cadeia;
 - Enviar as operações que modificam a lista (como `add` e `remove`) para o servidor que está na cabeça da cadeia;
 - Enviar as operações que consultam a lista (como `get_by_year`, `get_by_marca`, `size`, etc.) para o servidor que está na cauda da cadeia;
 - Fazer watch no ZooKeeper de forma a ser notificado de alterações na cadeia de replicação e ligar-se a novos servidores (na cabeça ou na cauda da cadeia), se estes tiverem mudado.

Como nos projetos anteriores, espera-se uma grande fiabilidade do servidor e do cliente, sendo para tal necessário tratar todas as condições de erro e garantir uma correta gestão da memória, evitando *memory leaks* ou acesso a zonas de memória inválidas. É importante notar que em sistemas cliente-servidor os servidores ficam em funcionamento permanente durante muito tempo (dias, meses ou até anos) e não é suposto que parem (*crash*) por causa de erros.

2 Descrição Detalhada

O objetivo específico da 4ª Fase do Projeto é desenvolver um sistema de Replicação em Cadeia (*Chain Replication*) [4] com múltiplos clientes e servidores. Para tal, para além de aproveitarem o código desenvolvido nas fases anteriores do projeto, os alunos devem fazer o uso de novas técnicas ensinadas nas aulas, incluindo o serviço ZooKeeper [5] para coordenação de sistemas distribuídos.

A figura a seguir ilustra a arquitetura final do sistema a desenvolver.

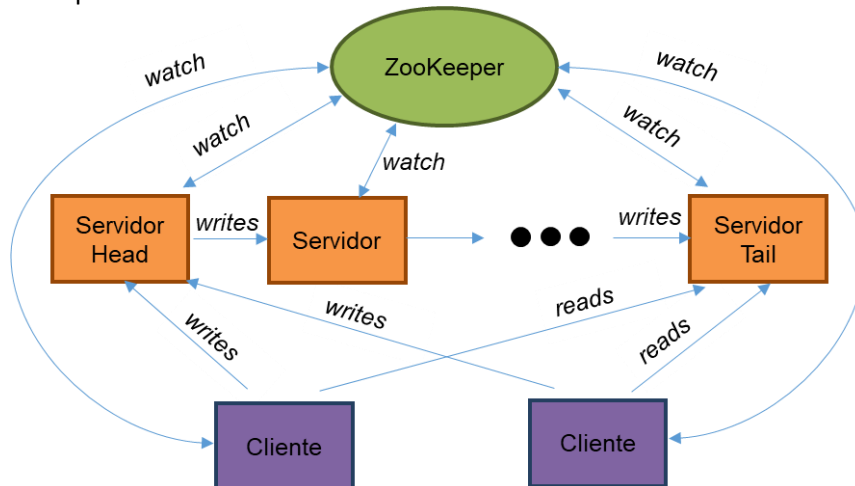


Figura 1 - Arquitetura geral da 4ª Fase do Projeto

Num modelo de replicação em cadeia, os servidores ligam-se entre si formando uma sequência, i.e. cada servidor apenas comunica com um outro servidor, que é o seu sucessor na cadeia. Por outro lado, os clientes têm de ter duas ligações: uma ao servidor na cabeça da cadeia, e outra ao servidor na cauda (já veremos porquê).

Para garantir que todos os servidores têm cópias iguais da lista (i.e., garantem a consistência dos dados), os clientes têm de enviar todas as operações que modificam a lista (como `add` e `remove`) para o servidor que está à cabeça da cadeia, sendo estas depois propagadas de forma síncrona (ou seja, ficando à espera da resposta, tal como é feito na função `network_send_receive()`) pelos servidores seguintes, até chegarem ao servidor que está na cauda. Assim, quando um servidor recebe uma operação de modificação, executa-a localmente e depois envia essa mesma operação para o seu sucessor (caso não esteja na cauda). As duas operações (local e remota) têm de ser feitas de forma atômica, ou seja, enquanto ambas não forem concluídas, nenhuma outra operação de modificação pode ser executada pelo servidor.

Por outro lado, os clientes enviam as operações que consultam a lista (como `get_by_year`, `get_by_marca`, `size`, etc.) para o servidor que está na cauda. Dessa forma, as leituras são sempre feitas sobre o servidor que possui o estado mais atualizado de toda a cadeia.

A ideia é que existam sempre pelo menos dois servidores ativos, para que a falha de um servidor possa ser tolerada. Se um servidor falhar, ficará sempre pelo menos um servidor ativo, não se perdendo o conteúdo da lista. Deverá ser possível iniciar novos servidores, fazendo com que estes se integrem na cadeia, ficando na cauda. Para se integrar, um novo servidor tem de começar por obter uma cópia atual da lista e, só depois, poderá começar a atender aos pedidos dos clientes.

Neste projeto, vamos considerar que as falhas dos servidores apenas podem acontecer quando não existem operações pendentes nos servidores (ou seja, não há clientes a fazer operações quando ocorre uma falha) e vamos também considerar que quando um novo servidor é iniciado também não há operações a serem executadas. Graças a estas considerações, a solução que tem de ser implementada acaba por ser simplificada.

2.1 ZooKeeper

Um elemento central na arquitetura desta fase do projeto é o ZooKeeper, que será usado para guardar a informação sobre os servidores que estão ativos e que avisará todos os nós, clientes ou servidores, sempre que algum servidor for adicionado ou falhar. Assim, o ZooKeeper permitirá manter uma visão completa e atualizada do sistema, não só sobre os servidores ativos, mas também sobre seus endereços IP e portos. Quando se iniciam os clientes, apenas será necessário passar como argumento a localização do ZooKeeper (IP:porto do ZooKeeper). Tanto os clientes como os servidores terão de se ligar e manter uma ligação ao ZooKeeper, interagindo com este através das funções disponibilizadas na API do ZooKeeper.

O ZooKeeper pode ser descrito como um serviço (eventualmente replicado, embora neste projeto se utilize apenas um servidor ZooKeeper) que armazena a informação organizada de forma hierárquica em nós que são designados **ZNodes**.

A imagem na página seguinte representa uma possível solução para se guardar informação no ZooKeeper sobre os servidores ativos. Nesta figura existem dois tipos de ZNodes: a `/chain` e os seus filhos `/node`. A `/chain` é um ZNode normal. É criada pelo primeiro servidor que se ligar ao ZooKeeper, se este ainda não existir, e deve continuar a existir mesmo que todos os servidores se deliguem do ZooKeeper. Os `/node` são os ZNodes filhos de `/chain`. Existe um `/node` para cada servidor do nosso sistema. Quando um servidor é iniciado e contacta o ZooKeeper, ele pede para criar um novo ZNode filho de `/chain` (i.e. o seu `/node`) com o seu `IP:porto` como conteúdo. O IP e o porto

servirão para outros servidores e clientes se poderem ligar a ele.

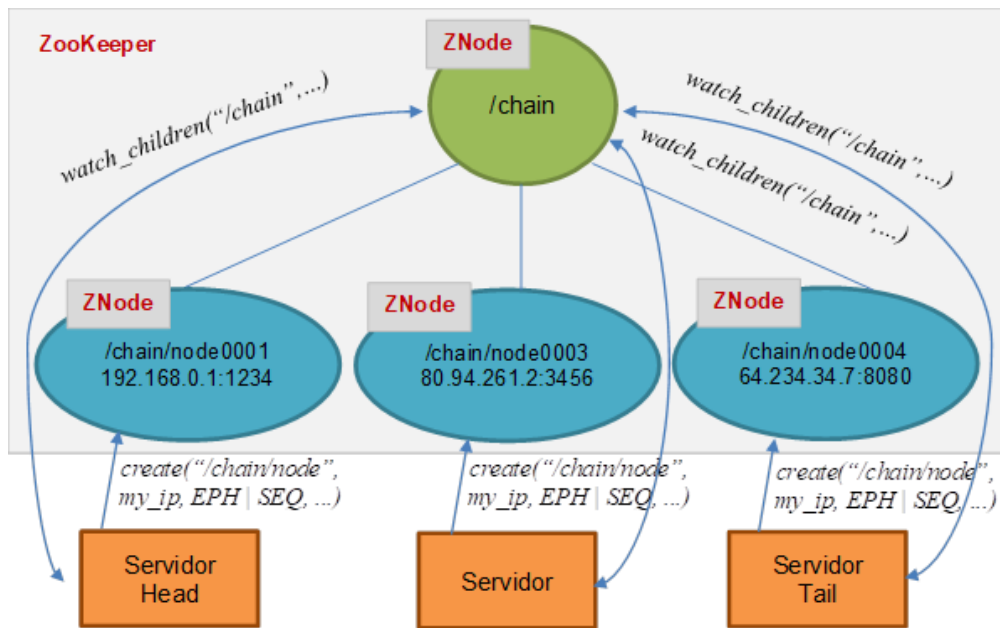


Figura 2 - Modelo de dados do ZooKeeper para Chain Replication.

Como queremos uma ordenação global entre os servidores para que exista sempre um (e apenas um) servidor *head* e um *tail*, vamos **criar os node como ZNodes sequenciais**. Assim, o ZooKeeper atribui um número de sequência único e crescente a cada novo `/node` que é criado. Quando se obtém a listagem de filhos de `/chain`, podemos ordenar os mesmos por ordem lexicográfica.

Como também pretendemos lidar com falhas dos servidores e detetar as mesmas de forma automática, vamos adicionalmente **criar os /node como ZNodes efémeros**. Assim, se um servidor falhar, o ZooKeeper deteta que a ligação entre os dois foi interrompida e remove o seu `/node` da lista de filhos de `/chain`, **notificando todos os servidores e clientes que fizeram watch aos filhos de /chain**. Isto significa que todos os servidores e clientes, quando são iniciados e se ligam ao ZooKeeper, devem indicar uma função que servirá como *callback* para receber estas notificações. E quando chamam uma função da API para ler dados do ZooKeeper, devem indicar ao ZooKeeper que querem ser notificados se a informação lida for alterada (ou seja, colocam o `watch` no ZNode lido).

2.2 Mudanças a efetuar no servidor

O servidor passa a guardar:

- uma ligação ao ZooKeeper;
- o identificador do seu `/node` no ZooKeeper;
- o identificador do `/node` do próximo servidor na cadeia de replicação, assim como uma *socket* para a comunicação com o mesmo. Dado que o servidor passa a assumir o papel de cliente relativamente ao seu sucessor (caso tenha sucessor), pode-se utilizar a estrutura `rlist_t` (modificando-a, se necessário, para guardar novas informações) bem como as funções definidas em `client_stub.c/.h` para comunicar com o servidor seguinte.

Novos passos a implementar na lógica do servidor quando este é iniciado:

- Ligar ao ZooKeeper;
- Criar um ZNode efémero sequencial no ZooKeeper, filho de `/chain`;
- Guardar o id atribuído ao ZNode pelo ZooKeeper;
- Obter e fazer `watch` aos filhos de `/chain`;
- Ver se existe um nó sucessor de entre os filhos de `/chain`, ou seja, um node com o identificador mais alto a seguir ao próprio identificador; Se existir, obter o conteúdo do Znode desse servidor (ou seja, `IP:porto`) ligando-se a ele como `next_server`. Caso contrário, `next_server` ficará `NULL`, o que significa que este servidor será a cauda da cadeia;
- Ver se existe um nó antecessor de entre os filhos de `/chain`, ou seja, o servidor com o identificador mais baixo antes do próprio identificador. Se existir, obter o conteúdo do ZNode desse servidor (`IP:porto`) e ligar-se temporariamente a ele para obter uma cópia atual da lista. Para isso, o servidor deve usar as operações remotas disponíveis (por exemplo, `get_list_ordered_by_year` ou `get_model_list`) e, para cada

carro obtido, inserir o elemento na lista local através da operação `add`. No final, a ligação ao servidor antecessor deve ser encerrada. Este passo serve para a sincronização inicial do estado da aplicação.

Nota: As inserções realizadas durante esta fase de sincronização inicial **não devem ser registadas no ficheiro de log**, uma vez que não correspondem a pedidos de clientes, mas apenas à recuperação interna do estado do servidor.

Adicionalmente:

- Quando a função de *callback* é chamada devido ao `watch` de filhos de `/chain` ter sido ativado, voltar a obter a lista de filhos de `/chain` (não esquecendo de ativar novamente o `watch`) para saber se o nó sucessor foi alterado novamente; caso tenha sido (porque o sucessor falhou, ou porque não havia sucessor e passou a haver), deve-se estabelecer uma ligação a este novo servidor e atualizar `next_server`;
- Quando receber uma operação que modifique a lista (por exemplo, `add` ou `remove`), o servidor deve executá-la localmente e, se não estiver na cauda da cadeia, executar também essa operação remotamente no servidor sucessor. Estas duas operações (execução local e execução no sucessor) têm de ser realizadas de forma atómica, ou seja, não podendo ser executada qualquer outra operação de modificação enquanto esta não estiver concluída.

Neste novo modelo, o executável `list_server` passa a receber o IP:porto do ZooKeeper, além do seu porto TCP. Todos os servidores deverão ser executados mantendo a mesma estrutura de dados (a lista ligada), garantindo assim que o estado da aplicação é corretamente replicado ao longo da cadeia.

2.3 Mudanças a efetuar no cliente

O cliente passa a ligar-se ao ZooKeeper e a dois servidores: o servidor head e o servidor tail da cadeia de replicação. Para tal, podem ser utilizadas duas cópias da estrutura `rlist_t`: uma para guardar os dados da ligação ao servidor head e outra para guardar os dados da ligação ao servidor tail. Se existir apenas um servidor ativo, este assumirá simultaneamente os papéis de head e tail, e o cliente manterá duas ligações ao mesmo servidor.

Novos passos a implementar na lógica do cliente quando este é iniciado:

- Ligar ao ZooKeeper;
- Obter e fazer `watch` aos filhos de `/chain`;
- Dos filhos de `/chain`, obter o IP:porto do servidor registado com o identificador mais baixo e do servidor que tem o identificador mais alto, guardá-los como *head* e *tail* respetivamente, ligando-se a eles;

Adicionalmente:

- Quando a função de *callback* é chamada devido ao `watch` de filhos de `/chain` ter sido ativado, voltar a obter a lista de filhos de `/chain` (não esquecendo de ativar novamente o `watch`) para saber se algum dos servidores *head* ou *tail* foi alterado, ligando-se ao novo servidor caso necessário;
- O cliente passa a enviar:
 - Pedidos que modificam a lista (por exemplo, `add` e `remove`) são enviados para o servidor *head*, sendo depois propagados ao longo de toda a cadeia;
 - Pedidos de consulta (como `get_by_year`, `get_by_marca`, etc.) são enviados para o servidor *tail*.

Neste modelo, o IP e o porto introduzidos pelo utilizador no comando `cliente_list` passam a corresponder apenas ao IP e porto do ZooKeeper.

2.4 Nota sobre o ficheiro de log

Tal como na 3.ª fase, cada servidor mantém o seu próprio ficheiro de log, onde regista os pedidos que processa. Com a replicação em cadeia, cada servidor passa a registar apenas os pedidos que recebe diretamente: o head regista todas as operações de modificação, o tail regista todas as operações de consulta, e os servidores intermédios apenas registam operações de modificação propagadas.

3 Makefile

Os alunos deverão utilizar o `Makefile` da 3ª Fase do Projeto, atualizando-o para compilar novo código, se necessário.

4 Entrega

A entrega da 4ª Fase do Projeto tem de ser feita de acordo com as seguintes regras:

1. Colocar todos os ficheiros do projeto, bem como o ficheiro `README` mencionado abaixo, num ficheiro com compressão no formato ZIP. O nome do ficheiro será **SD-XX-projeto4.zip** (XX é o número do grupo).

2. Submeter o ficheiro **SD-XX-projeto4.zip** na página da disciplina no moodle da FCUL, utilizando a atividade disponibilizada para tal. Apenas um dos elementos do grupo deve submeter, considerando-se apenas a submissão mais recente no caso de existirem várias.

O ficheiro ZIP deverá conter uma diretoria cujo nome é **SD-XX**, onde **XX** é o número do grupo. Nela serão colocados:

- o ficheiro **README**, onde os alunos podem incluir informações que julguem necessárias (e.g., limitações na implementação);
- diretorias adicionais, nomeadamente:
 - **include**: para armazenar os ficheiros `.h`;
 - **source**: para armazenar os ficheiros `.c`;
 - **lib**: para armazenar bibliotecas;
 - **object**: para armazenar os ficheiros objeto;
 - **binary**: para armazenar os ficheiros executáveis.
- um ficheiro **Makefile** que permita a correta compilação de todos os ficheiros entregues. Não devem ser incluídos no ficheiro ZIP os ficheiros objeto (`.o`) ou executáveis que são construídos pelo `Makefile`. Caso sejam usados os ficheiros objeto da 1ª fase do projeto disponibilizados aos grupos, estes **devem** ser incluídos no ficheiro ZIP.

Na entrega do trabalho, é ainda necessário ter em conta que:

- **Se não for incluído um Makefile, se o mesmo não compilar os ficheiros fonte, ou se houver erros de compilação (isto é, se não forem criados os ficheiros objeto e executáveis), o trabalho é considerado nulo.** Na página da disciplina, no Moodle, podem encontrar documentos do utilitário `make` e dos ficheiros `Makefile` (cortesia da disciplina de Sistemas Operativos).
- Todos os ficheiros entregues devem começar com um cabeçalho com três ou quatro linhas de comentários a dizer o número do grupo e o nome e número dos seus elementos.
- Os programas são testados no ambiente dos laboratórios de aulas, pelo que se recomenda que os alunos testem os seus programas neste mesmo ambiente.

O prazo de entrega é dia 9/12/2025 até às 23:59 horas.

Após esta data, a submissão do trabalho através do Moodle deixará de ser permitida.

5 Autoavaliação de contribuições

Cada aluno tem de preencher no Moodle um formulário de autoavaliação das contribuições individuais de cada elemento do grupo para o projeto. Por exemplo, se todos os elementos colaboraram de forma idêntica, bastará que todos indiquem que cada um contribuiu 33%. Aplicam-se as seguintes regras e penalizações:

- Alunos que não preencham o formulário até a data limite de entrega do projeto **sofrem uma penalização na nota de 20%.**
- Caso existam assimetrias significativas entre as respostas de cada elemento do grupo, o grupo poderá ser chamado para as explicar.
- Se as contribuições individuais forem diferentes, isso será refletido na nota de cada elemento do grupo, levando à atribuição de notas individuais diferentes.

O prazo de preenchimento deste formulário é o mesmo que a entrega do projeto (9/12/2025, até às 23:59 horas).

6 Plágio

Não é permitido aos alunos partilharem códigos com soluções, ainda que parciais, de nenhuma parte do projeto com outros alunos (nem através do Fórum da disciplina, nem por qualquer outro meio). Além disso, todos os códigos serão testados por um verificador de plágio. Caso alguma irregularidade seja encontrada, os projetos de todos os alunos envolvidos serão anulados e o caso será reportado aos órgãos responsáveis em Ciências@ULisboa.

Chamamos a atenção para o facto das plataformas generativas baseadas em Inteligência Artificial (e.g., o ChatGPT e o GitHub Co-Pilot) (1) gerarem um número limitado de soluções diferentes para o mesmo problema, (2) podem não resolver corretamente as alíneas descritas no projeto, e (3) incluem padrões característicos deste tipo de ferramenta, as quais podem vir a ser detetáveis pelos verificadores de plágio. Desta forma, recomendamos fortemente que os alunos não submetam trechos de código gerados por este tipo de ferramenta a fim de evitar riscos desnecessários.

Por fim, é responsabilidade de cada aluno garantir que a sua *home*, as suas diretorias e os seus ficheiros de código estão protegidos contra a leitura de outras pessoas (que não o utilizador dono dos mesmos). Por exemplo, se os ficheiros estiverem gravados na sua área de aluno nos servidores de Ciências@ULisboa, então todos os itens mencionados anteriormente devem ter as permissões de acesso `700`. Se os ficheiros estiverem no GitHub, garantam

que o conteúdo do vosso repositório não esteja visível publicamente. Caso contrário, a sua participação num eventual plágio será considerada ativa.

7 Bibliografia

[1] B. W. Kernighan, D. M. Ritchie, C Programming Language, 2nd Ed, Prentice-Hall, 1988.

[2] Wikipedia. Linked List. https://en.wikipedia.org/wiki/Linked_list

[3] <https://developers.google.com/protocol-buffers/docs/overview>

[4] R. V. Renesse and F. B. Schneider. Chain Replication for Supporting High Throughput and Availability. OSDI. Vol. 4. No. 91–104. 2004.

[5] <https://zookeeper.apache.org/>